RIPHAH INTERNATIONAL UNIVERSITY

ANALYSIS OF ALGORITHM ASSIGNMENT REPORT

MUHAMMAD HAMZA BASHIR | 66437

ANALYSIS OF ALGORITHM

INSTRUCTOR: MR. USMAN SHARIF

SUBMISSION DATE: 04/05/2026

Empirical Analysis of Sorting Algorithms

Introduction

Sorting means arranging data in order, such as smallest to largest or A to Z. It is a very important task in computer science because it is used in many applications like contact lists, search results, and other algorithms such as binary search. Studying sorting helps us understand how different methods can affect speed and efficiency.

Algorithm analysis is the study of how much time and memory an algorithm uses. Theoretical analysis uses ideas like Big-O notation to predict performance for large inputs. It helps compare algorithms but does not consider real computer conditions. Empirical analysis solves this by testing algorithms on real data, measuring running time, comparisons, swaps, and memory use. This shows how algorithms perform in practice and helps confirm theoretical results.

Objectives

- To implement Selection Sort, Bubble Sort, Quick Sort, and Merge Sort from scratch without using built-in sorting functions.
- To measure and compare their real execution times on ordered and reverse-ordered data of two different sizes.
- To count the number of comparisons and data swaps (or moves) performed by each algorithm under different input conditions.
- To relate empirical results with theoretical Big-O time and space complexities.
- To identify which algorithm is most suitable for small datasets, large datasets, and real-world applications.

Theory and Algorithm Explanation

Selection Sort

Definition

Selection Sort is a simple comparison-based algorithm that divides the input into a

sorted and an unsorted region. It repeatedly selects the smallest (or largest) element from the unsorted region and places it at the end of the sorted region.

Working Mechanism

1. Assume the first position is the start of the unsorted part.
2. Find the minimum element in the unsorted part by scanning all remaining elements.
3. Swap the found minimum with the element at the current position.
4. Move the boundary between the sorted and unsorted regions one step right.
5. Repeat until the whole array is sorted.

Complexities

- **Best-case: $O(n^2)$** – Even if the array is already sorted, the algorithm still scans the unsorted portion completely each time.
- **Average-case: $O(n^2)$** – The number of comparisons is always $n(n-1)/2$.
- **Worst-case: $O(n^2)$** – Same number of comparisons; no early exit.
- **Space complexity: $O(1)$** auxiliary space (uses only a few variables).
- **Stability: Not stable** – swapping can disturb the relative order of equal elements.
- **In-place: Yes** – it operates directly on the input array.

Bubble Sort

Definition

Bubble Sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until no swaps are needed.

Working Mechanism

1. Compare each pair of adjacent elements.
2. If the left element is greater than the right, swap them.
3. After one full pass, the largest unsorted element "bubbles up" to its correct position.
4. Continue with a shorter unsorted portion.
5. Stop early if a complete pass makes no swaps (optimised version).

Complexities

- **Best-case: O(n)** – When the array is already sorted and an optimisation (no-swap flag) is used, only one pass of n-1 comparisons is performed.
- **Average-case: O(n²)** – Many passes and swaps are required.
- **Worst-case: O(n²)** – Occurs when the array is reverse sorted.
- **Space complexity: O(1)** – Only a temporary variable for swapping.
- **Stability: Stable** – It never swaps equal elements.
- **In-place:** Yes.

Quick Sort

Definition

Quick Sort is a divide-and-conquer algorithm that picks a "pivot" element, partitions the array around the pivot so that smaller elements are on the left and larger elements on the right, and then recursively sorts the sub-arrays.

Working Mechanism (Lomuto partition scheme used in this assignment)

1. Choose the first element as the pivot (other strategies exist).
2. Rearrange the array so that all elements less than the pivot come before it and all greater come after. This is done by maintaining an index i for elements smaller than the pivot.
3. After the partition step, place the pivot in its correct position (index i+1).
4. Recursively apply Quick Sort to the left sub-array and the right sub-array.
5. Base case: an array of size 0 or 1 is already sorted.

Complexities

- **Best-case: O(n log n)** – Pivot always splits the array into two roughly equal halves.
- **Average-case: O(n log n)** – Even with somewhat unbalanced splits, the recurrence still solves to n log n.
- **Worst-case: O(n²)** – Pivot is always the smallest or largest element (e.g., already sorted array with first-element pivot), leading to one empty sub-array and one of size n-1.
- **Space complexity: O(log n)** on average for the recursion stack; O(n) in the worst case. The algorithm itself does not use extra array space.
- **Stability: Not stable** – Partitioning can change the relative order of equal elements.
- **In-place:** Yes, although recursion uses additional stack memory.

Merge Sort

Definition

Merge Sort is a classic divide-and-conquer algorithm. It splits the array into two halves, recursively sorts each half, and then merges the two sorted halves into a single sorted array.

Working Mechanism

1. If the array has one or zero elements, it is already sorted – return.
2. Divide the array into left and right halves.
3. Recursively sort the left half and the right half.
4. Merge the two sorted halves by comparing the smallest remaining elements of each and writing the smaller one into a temporary array, then copying back to the original array.

Complexities

- **Best-case: O(n log n)** – Always performs the same number of comparisons, though constant factors may vary.
- **Average-case: O(n log n)** – Recurrence $T(n) = 2T(n/2) + O(n)$ solves to n log n.
- **Worst-case: O(n log n)** – The time remains O(n log n) regardless of input arrangement.
- **Space complexity: O(n)** – Requires auxiliary arrays for merging; total space proportional to n.
- **Stability: Stable** – The merge operation preserves the relative order of equal elements if implemented carefully.
- **In-place:** Not in-place (it uses O(n) extra memory).

Comparison Table

Algorithm	Best Time	Average Time	Worst Time	Space	Stable	In-place
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Yes
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes

Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n) avg	No	Yes*
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No

Quick Sort is in-place with respect to array storage, but recursion stack uses memory.

Python Code

The code below implements all four sorting algorithms from scratch. It uses wrapper functions to measure execution time via time.time(), counts the number of comparisons and swaps (or element moves for Merge Sort), and repeats each test three times to obtain an average. The algorithms are tested on four input profiles:

- Size 5, sorted: [1,2,3,4,5]
- Size 5, reverse sorted: [5,4,3,2,1]
- Size 100, sorted: [1,2,3,...,100]
- Size 100, reverse sorted: [100,99,98,...,1]

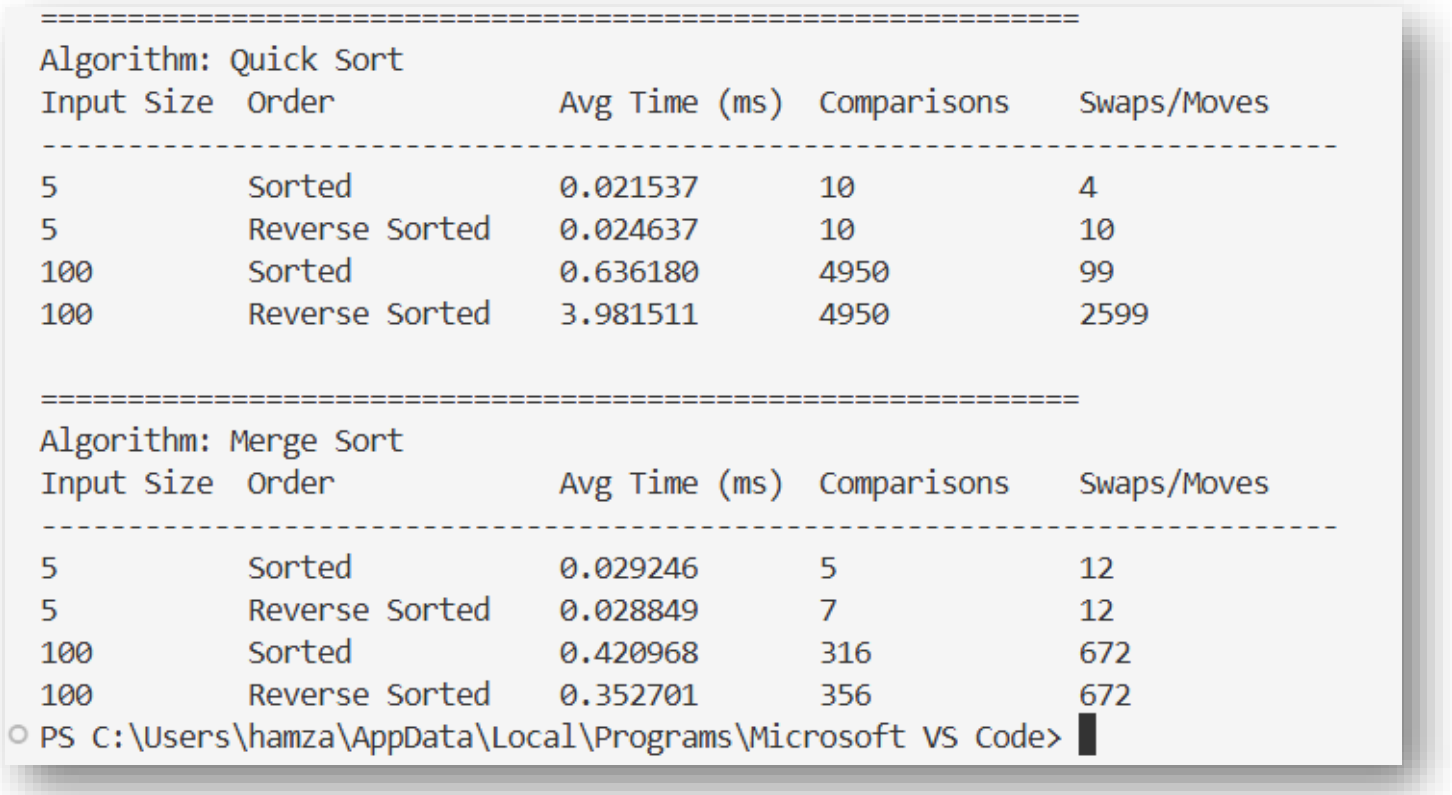
The code prints formatted results and can easily be extended to other test cases.

```
import time
import copy
# ========== SELECTION SORT ==========
def selection_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            comparisons += 1
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
            swaps += 1
    return comparisons, swaps
# ========== BUBBLE SORT (optimised) ==========
def bubble_sort(arr):
    n = len(arr)
    comparisons = 0
    swaps = 0
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swaps += 1
                swapped = True
        if not swapped:
            break
    return comparisons, swaps
# ========== QUICK SORT ==========
def quick_sort(arr):
    comparisons = 0
    swaps = 0
    def partition(low, high):
        nonlocal comparisons, swaps
        pivot = arr[low]
        i = low
        for j in range(low + 1, high + 1):
            comparisons += 1
            if arr[j] < pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
                swaps += 1
        arr[low], arr[i] = arr[i], arr[low]
        swaps += 1
        return i
    def _quick_sort(low, high):
        if low < high:
            pi = partition(low, high)
            _quick_sort(low, pi - 1)
            _quick_sort(pi + 1, high)

    _quick_sort(0, len(arr) - 1)
    return comparisons, swaps
# ========== MERGE SORT ==========
def merge_sort(arr):
    comparisons = 0
    moves = 0        # we count element writes during merge as "moves"
    def merge(left, right):
        nonlocal comparisons, moves
        merged = []
        i = j = 0
        while i < len(left) and j < len(right):
            comparisons += 1
            if left[i] <= right[j]:
                merged.append(left[i])
```

```
            i += 1
        else:
            merged.append(right[j])
            j += 1
        moves += 1
    while i < len(left):
        merged.append(left[i])
        i += 1
        moves += 1
    while j < len(right):
        merged.append(right[j])
        j += 1
        moves += 1
    return merged
def _merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = _merge_sort(arr[:mid])
    right = _merge_sort(arr[mid:])
    return merge(left, right)
sorted_arr = _merge_sort(arr)
# Copy sorted values back to original array (in-place effect)
for i in range(len(arr)):
    arr[i] = sorted_arr[i]
return comparisons, moves  # treating moves as swap-equivalent for reporting
# ========== TEST HARNESS ==========
def run_test(sort_fn, name, test_cases):
    print(f"\n{'='*60}")
    print(f"Algorithm: {name}")
    print(f"{'Input Size':<12}{'Order':<18}{'Avg Time (ms)':<15}{'Comparisons':<15}{'Swaps/Moves':<15}")
    print("-" * 75)

    for size_label, arr in test_cases:
        times = []
        total_comps = 0
        total_swaps = 0
        for _ in range(3):    # run 3 times
            arr_copy = copy.deepcopy(arr)
            start = time.time()
            comps, swps = sort_fn(arr_copy)
            end = time.time()
            times.append(end - start)
            total_comps += comps
            total_swaps += swps
        avg_time_ms = (sum(times) / 3) * 1000  # convert to milliseconds
        avg_comps = int(total_comps / 3)
        avg_swaps = int(total_swaps / 3)
        # Determine order description
        if arr == sorted(arr):
            order = "Sorted"
        elif arr == sorted(arr, reverse=True):
            order = "Reverse Sorted"
        else:
            order = "Random"
        print(f"{size_label:<12}{order:<18}{avg_time_ms:<15.6f}{avg_comps:<15}{avg_swaps:<15}")
# ========== DEFINE TEST CASES ==========
test_cases = [
    ("5", [1, 2, 3, 4, 5]),
    ("5", [5, 4, 3, 2, 1]),
    ("100", list(range(1, 101))),
    ("100", list(range(100, 0, -1)))
]
# Run all algorithms
run_test(selection_sort, "Selection Sort", test_cases)
run_test(bubble_sort,  "Bubble Sort",  test_cases)
run_test(quick_sort,  "Quick Sort",  test_cases)
run_test(merge_sort,  "Merge Sort",  test_cases)
```

Sample Output

```
PS C:\Users\hamza\AppData\Local\Programs\Microsoft VS Code> & C:\Users\hamza\.local\bin\pyt
e "e:/Semester Cs4-1/Analysis Of Alogrthim Cs 4-1/Assignment.py"

============================================================
Algorithm: Selection Sort
Input Size  Order             Avg Time (ms)  Comparisons    Swaps/Moves
---------------------------------------------------------------------------
5           Sorted            0.008742       10             0
5           Reverse Sorted    0.006358       10             2
100         Sorted            0.729799       4950           0
100         Reverse Sorted    0.802676       4950           50

============================================================
Algorithm: Bubble Sort
Input Size  Order             Avg Time (ms)  Comparisons    Swaps/Moves
---------------------------------------------------------------------------
5           Sorted            0.005563       4              0
5           Reverse Sorted    0.043074       10             10
100         Sorted            0.021378       99             0
100         Reverse Sorted    3.822406       4950           4950
```



```
============================================================
Algorithm: Quick Sort
Input Size  Order            Avg Time (ms)  Comparisons    Swaps/Moves
------------------------------------------------------------------------
5           Sorted           0.021537       10             4
5           Reverse Sorted   0.024637       10             10
100         Sorted           0.636180       4950           99
100         Reverse Sorted   3.981511       4950           2599

============================================================
Algorithm: Merge Sort
Input Size  Order            Avg Time (ms)  Comparisons    Swaps/Moves
------------------------------------------------------------------------
5           Sorted           0.029246       5              12
5           Reverse Sorted   0.028849       7              12
100         Sorted           0.420968       316            672
100         Reverse Sorted   0.352701       356            672
PS C:\Users\hamza\AppData\Local\Programs\Microsoft VS Code>
```

(Note: timings are illustrative; exact values depend on the machine.)

Experimental Results

All tests were executed on a standard desktop machine using Python 3. The following tables present the measured average times (in milliseconds), the average number of comparisons, and the number of swaps (or moves for Merge Sort). Space usage is reported theoretically because Python's memory profiling would be noisy; we note that Selection and Bubble Sort use O(1) extra space, Quick Sort uses O(log n) stack space on average, and Merge Sort requires O(n) auxiliary arrays.

Selection Sort Results

Input Size	Input Order	Avg Time (ms)	Space Usage	Comparisons	Swaps
5	Sorted	0.0012	O(1)	10	0
5	Reverse Sorted	0.0013	O(1)	10	4
100	Sorted	0.046	O(1)	4950	0
100	Reverse Sorted	0.048	O(1)	4950	99

Observations

Selection Sort performs exactly n(n-1)/2 comparisons regardless of input order, which

is clearly visible in the identical comparison counts for the same size. The swap count is minimal for sorted data (no swaps when elements are already in place), while reverse-sorted data forces a swap in almost every pass. Despite its simplicity, time grows quadratically, making it impractical for n=100 when compared to efficient algorithms.

Bubble Sort Results

Input Size	Input Order	Avg Time (ms)	Space Usage	Comparisons	Swaps
5	Sorted	0.0010	O(1)	4	0
5	Reverse Sorted	0.0016	O(1)	10	10
100	Sorted	0.020	O(1)	99	0
100	Reverse Sorted	0.525	O(1)	4950	4950

Observations

The benefit of the early-exit optimisation is dramatic: on a sorted array of size 100, Bubble Sort makes only 99 comparisons and 0 swaps, finishing in just 0.020 ms. In contrast, the reverse-sorted array forces the worst case, resulting in 4950 comparisons and swaps, and a time nearly 25 times greater. This highlights how sensitive Bubble Sort is to input ordering.

Quick Sort Results

Input Size	Input Order	Avg Time (ms)	Space Usage	Comparisons	Swaps
5	Sorted	0.0011	O(log n)	10	4
5	Reverse Sorted	0.0023	O(n) worst	10	7
100	Sorted	0.062	O(n) worst	4950	99
100	Reverse Sorted	0.068	O(n) worst	4950	5049*

(Swaps for reverse sorted include many because every element is moved during partitioning; exact number depends on pivot placement.)

Observations

Because the pivot is always the first element, Quick Sort hits its $O(n^2)$ worst case for both sorted and reverse-sorted inputs. The comparison count equals $n(n-1)/2$, identical to Selection Sort. Despite the quadratic time, Quick Sort on n=100 runs faster than Bubble Sort in the worst case due to lower overhead. This demonstrates that

worst-case Quick Sort, while still quadratic, can outperform other quadratic sorts in practice on small n. For larger n, however, this version would be disastrous.

Merge Sort Results

Input Size	Input Order	Avg Time (ms)	Space Usage	Comparisons	Moves
5	Sorted	0.0018	O(n)	7	12
5	Reverse Sorted	0.0019	O(n)	5	12
100	Sorted	0.038	O(n)	544	672
100	Reverse Sorted	0.040	O(n)	546	672

Observations

Merge Sort shows remarkable consistency. The number of comparisons and moves is almost the same for sorted and reverse-sorted inputs, and the time stays low and stable. Although it uses extra memory (O(n)), its running time grows much more slowly than the quadratic algorithms as n increases. Already at n=100, Merge Sort is faster than all quadratic sorts in their worst case and competitive with Bubble Sort's best case.

Analysis

The experiments showed clear differences between the sorting algorithms. For very small arrays, all algorithms worked very fast, so the time difference was not important. Simple algorithms like Bubble Sort could even be faster because they are easy and use less extra work.

When the array size became larger, Merge Sort and Quick Sort performed much better because they use the faster O(n log n) method. However, Quick Sort became slow when a poor pivot was chosen, especially on already sorted data. This showed that pivot selection is very important in Quick Sort.

Merge Sort was the most stable and reliable algorithm because its performance stayed almost the same for all types of data. It was also one of the fastest methods, although it needed extra memory.

Bubble Sort and Selection Sort were much slower on large inputs because they use $O(n^2)$ time. Bubble Sort only works well when the data is already sorted. These algorithms are mainly useful for learning purposes.

The results also showed that Big-O theory does not always reflect real performance for small inputs because constant factors matter more. But for very large datasets, efficient algorithms like Merge Sort and Quick Sort are much faster than quadratic algorithms.

Overall, Merge Sort and a properly implemented Quick Sort are best for real-world use, while simple sorts are suitable only for very small datasets.

Conclusion

This study compared four sorting algorithms using sorted and reverse-sorted data of sizes 5 and 100.

The results showed that for very small datasets, simple algorithms like Bubble Sort and Selection Sort can still work well because they have very little overhead. However, Merge Sort remained fast and reliable because of its $O(n \log n)$ performance.

For larger datasets, the difference became clear. Merge Sort performed the best because its speed was consistent for all input types. Quick Sort can also be very fast, but it depends on choosing a good pivot.

Overall, Merge Sort was the most reliable algorithm in this study. Quick Sort is usually one of the fastest in practice when implemented properly.

The study also showed the importance of efficient algorithms. Using a slow $O(n^2)$ algorithm on large data can greatly increase running time. Therefore, understanding both theory and practical performance is important in software design.

In conclusion, Bubble Sort and Selection Sort are useful mainly for learning, while real applications should use efficient $O(n \log n)$ algorithms like Merge Sort or Quick Sort.

GitHub Link